CS 141 - Lab 5: Singly Linked Lists

Lab Objectives

By the end of this lab, you should be able to:

- Declare a `struct Node` for a singly linked list
- Build a list by repeatedly inserting at the **beginning**
- **Search** for a value in the list
- **Delete** the first node (remove head) and `free` it
- Print the list for debugging (traverse from `head`)

Exercise 1: Node type and print the list

Objective: Set up the list type and a function to display all values.

Task

1. Create `exercise1.c`
2. Include `<stdio.h>` and `<stdlib.h>`

Define:

```
struct Node {
    int data;
    struct Node *next;
};
```

3.
4. Implement `void print_list(struct Node *head)`:
    - Walk the list from `head` using a pointer `current`
    - Print each `data` (e.g., separated by spaces), then a newline
    - If the list is empty, print something like `(empty)` or an empty line
5. In `main`, set `struct Node *head = NULL;`, call `print_list(head);`

Expected output (example)

```
(empty)
```

Hints

- Loop pattern: `for (struct Node *c = head; c != NULL; c = c->next)`

Exercise 2: Insert at the beginning

Objective: Implement `insert_beginning` as on the intro page.

Task

1. Create `exercise2.c` (you may copy your `struct Node` and `print_list` from Exercise 1)

Implement:
`struct Node *insert_beginning(struct Node *head, int value);`

2.
    - Allocate a new node with `malloc(sizeof(struct Node))`
    - If `malloc` returns `NULL`, return `head` unchanged (the list stays the same)
    - Set `data`, link `next` to the old `head`, then **return** the new node as the new head
3. In `main`:
    - Start with `head == NULL`
    - Insert the values `3`, then `2`, then `1` (in that order) at the beginning, each time assigning the return value back to `head`
    - Call `print_list(head)` — you should see `1 2 3` (most recent insert is at the front)

Expected output (example)

```
1 2 3
```

Hints

- Call from `main`: `head = insert_beginning(head, 3);`

Exercise 3: Search

Objective: Find whether a value appears in the list.

Task

1. Create `exercise3.c`
2. Implement either:
    - `struct Node *find(struct Node *head, int value);`
      returns a pointer to the first node with that `data`, or `NULL` if not found
      or

- `int contains(struct Node *head, int value);`
  returns `1` if found, `0` if not

3. In `main`, build a small list (e.g. insert `10`, `20`, `30` at the beginning so print order is `30 20 10`), then test search for a value that exists and one that does not. Print clear messages.

Expected output (example)

```
List: 30 20 10
Found 20
Did not find 99
```

Exercise 4: Delete at the beginning

Objective: Remove the first node and free its memory.

Task

1. Create `exercise4.c`

Implement:
`struct Node *delete_beginning(struct Node *head);`

2.
   - If the list is empty (`head == NULL`), return `NULL`
   - Otherwise remove the first node, `free` it, and **return** the pointer to the new first node (same idea as the intro)
3. In `main`:
   - Insert a few values, print the list
   - Do `head = delete_beginning(head);` once, print again
   - Repeat `head = delete_beginning(head);` until the list is empty; print after each step or at the end
   - Verify `print_list` shows the correct sequence and that deleting from an empty list does not crash (`delete_beginning(NULL)` should return `NULL`)

Expected output (example)

```
Before: 5 4 3
After one delete: 4 3
After deletes: (empty)
```

Challenge: Tiny menu

Objective: Reuse your functions in a small interactive program.

Task

1. Create `challenge.c`
2. Keep `head` in `main`. Loop until the user quits, with a menu such as:
    - `1` — read an integer and **insert at beginning**
    - `2` — **print** the list
    - `3` — read an integer and report **found / not found**
    - `4` — **delete** at the beginning
    - `0` — **quit**
3. Use `scanf` (or your preferred input) for choices and values.
4. Whenever you insert or delete at the front, update `head` with the returned pointer: `head = insert_beginning(head, x);` and `head = delete_beginning(head);`.

Note

Fully freeing the whole list before exit is excellent practice;in the main lecture you will cover list teardown in detail. For this lab, deleting with option `4` until empty is enough.